MAKERERE UNIVERSITY

BSE4104: Emerging Trends in Software Engineering

8-Week AI-Native and Agentic Engineering Capstone

WEEK 4 WEEKLY PROGRESS REPORT: RAG AND TOOL/FUNCTION CALLING

Group: Group G

Project: University Student-Support Case Agent

Reporting period: 21–25 September 2026

Group leader: Stacey Ampaire

1. Work Completed Against Weekly Objectives

During Week 4, the group continued developing the University Student-Support Case Agent, focusing on retrieval-augmented generation (RAG), tool/function calling, and the controls required to keep the system within its approved scope.

The RAG implementation was developed to retrieve relevant information from an approved document collection before generating responses. The pipeline loaded 163 document chunks and generated an embedding matrix with dimensions of 163 × 384. Similarity-based retrieval was used to identify relevant context for student-support questions, including questions about registration.

The tool/function-calling work focused on enabling the model to request approved application functions rather than perform arbitrary operations. The two intended tools were get_case_status, for retrieving existing support-case information, and create_support_ticket, for creating simulated support tickets subject to application validation.

My individual responsibility was **failure and authorisation test evidence**. This involved examining how the application should respond to valid and invalid case identifiers, incomplete ticket requests, and requests for actions outside the project's authorised scope. The key objective was to demonstrate that tool calls are controlled by application logic and that the agent does not perform prohibited university-record changes.

2. Key Engineering Decisions

- **Retrieval-grounded responses:** RAG was used to retrieve relevant approved information rather than relying entirely on the model's general knowledge.

- **Deterministic tool execution:** Case lookup and ticket creation were implemented as explicit application functions, with validation handled by the application.
- **Synthetic case data:** The case-management demonstration used synthetic student-support records instead of depending on confidential university databases.
- **Restricted actions:** The system was designed not to make admissions, grading, fee, or disciplinary decisions, or to change official university records.
- **Failure-aware behaviour:** Unknown cases and invalid requests should produce clear failure responses rather than fabricated results or unauthorised actions.

3. Challenges and Responses

The RAG pipeline encountered model-provider availability problems during testing. Gemini returned a 503 error, while the Groq fallback encountered a 403 network/access-denial error in the recorded test. These failures prevented that particular run from demonstrating a complete successful answer-generation path. The retrieval stage had nevertheless loaded the document chunks and produced embeddings and similarity-ranked results.

The tool-calling work also required careful distinction between the model requesting an action and the application authorising and executing it. A model-generated tool call must not automatically be treated as permission to perform the action. The application therefore needs deterministic validation of required arguments, existing case identifiers, allowed categories, and prohibited operations.

These challenges informed the need for repeatable tests that capture both successful operations and safe failures.

4. Individual Contributions and Evidence

Member	Responsibility	Evidence to retain
Stacey Ampaire	Failure and authorisation test evidence	Test cases, execution outputs, and evidence of rejection or safe handling of invalid and out-of-scope requests
Judith Kemirembe	Tool/function-calling demonstration	Tool-call demonstration and recorded execution output
Desmond Cole	Tool catalogue and API	Tool descriptions, argument schemas, and validation

Aineamatsiko	schemas	requirements
Arnold Fred Kawuma	Architecture diagram	Diagram showing the model, application layer, approved tools, and their boundaries
Rachel Mirembe	Documentation and evidence coordination	Weekly report, repository evidence, and ClickUp task updates

Repository: Group G — University Student-Support Case Agent

Relevant evidence: RAG retrieval output, tool-calling execution logs, failure/authorisation test evidence, and the corresponding documentation in the repository.

ClickUp evidence: [Insert the relevant Week 4 task or board link]

5. Plan for Week 5

The next phase will focus on implementing a bounded, goal-directed agent workflow. The agent will use an explicit sequence of observing the student's request, checking case information, selecting an approved next action, observing the tool result, and stopping or recovering safely.

The group will define the agent's goal, approved tools, state, iteration limit, and stop conditions. Testing will include a successful multi-step workflow, an unknown-case failure with safe recovery, and an out-of-scope request. Execution traces will be retained to demonstrate the agent's behaviour and support evaluation.

Conclusion: Week 4 established important foundations for grounding responses in retrieved information and interacting with application functions under explicit constraints. The main outstanding concerns were model-provider reliability and the need for clear, repeatable evidence that tool execution remains authorised and bounded.